



Triggers em MySQL

Automação e controle de dados com triggers no MySQL

SM

por Salvador Melo



Objetivos da Aula



Entender triggers

Conceitos fundamentais



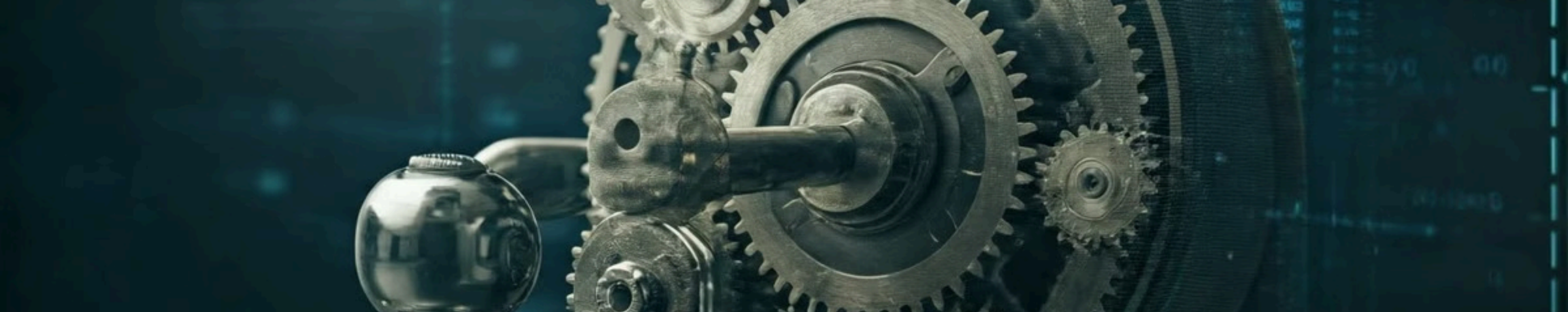
Aprender sintaxe

Tipos de triggers no MySQL



Implementar casos reais

Usando PlanoSaudeDB



O que são Triggers

Procedimentos automáticos

Código SQL que executa em resposta a eventos DML (INSERT, UPDATE, DELETE) em tabelas específicas do banco de dados

Auditoria e rastreamento

Registram alterações nos dados (quem, quando, valores antigos e novos) em tabelas de log para controle histórico

Validações complexas

Garantem integridade referencial e regras de negócio além das constraints básicas do MySQL, verificando condições antes da modificação

Sincronização de dados

Mantêm tabelas relacionadas atualizadas automaticamente, como saldos, históricos e estatísticas no PlanoSaudeDB

Tipos de Triggers



BEFORE INSERT

Executa antes da inserção de registros, permitindo validar dados ou modificar valores antes de serem gravados na tabela.



AFTER INSERT

Executa após a inserção bem-sucedida, ideal para criar logs de auditoria ou propagar dados para tabelas relacionadas.



BEFORE UPDATE

Executa antes da atualização de registros, permitindo validar alterações ou formatar dados antes de modificar a tabela.



AFTER UPDATE

Executa após a atualização, útil para registrar mudanças ou sincronizar dados entre tabelas dependentes.



BEFORE DELETE

Executa antes da exclusão de registros, permitindo verificar restrições ou criar backups antes da remoção dos dados.



AFTER DELETE

Executa após a exclusão bem-sucedida, frequentemente usado para manter histórico ou limpar tabelas relacionadas.

Sintaxe Geral

```
DELIMITER //  
CREATE TRIGGER nome_trigger  
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
ON nome_tabela  
FOR EACH ROW  
BEGIN  
    -- Instruções SQL  
END;  
// DELIMITER ;
```

Tabela para Auditoria



Criar tabela Log_Consultas

Para registrar alterações

2

Criar tabela cadastro

para inserir novos usuários



Definir colunas

id_log, id_consulta, acao, data_log, usuario



Configurar chave primária

id_log AUTO_INCREMENT

Database e Tabelas

```
create database auditoria;  
  
use auditoria;
```

```
create table log_consultas(  
  
id_log int auto_increment primary key,  
  
id_consulta int not null,  
  
acao varchar(200),  
  
data_log datetime,  
  
usuario varchar(200) );
```

```
CREATE TABLE Consultas (  
  
id_consulta INT AUTO_INCREMENT PRIMARY KEY,  
  
data_consulta date,  
  
descricao VARCHAR(255) );
```

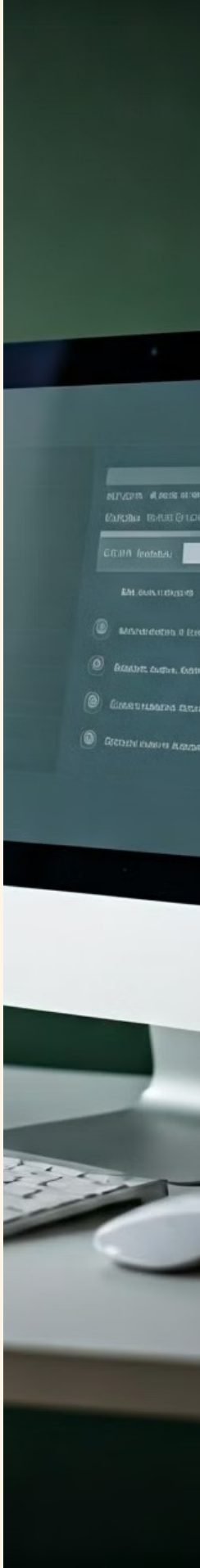
AFTER INSERT em Consultas

```
DELIMITER //  
  
CREATE TRIGGER trg_depois_insert_consulta  
  
AFTER INSERT ON Consultas  
  
FOR EACH ROW  
  
BEGIN  
  
INSERT INTO  
  
Log_Consultas (id_consulta, acao, data_log, usuario)  
  
VALUES (NEW.id_consulta, 'INSERT', NOW(), USER());  
  
END;  
  
// DELIMITER ;
```

```
select * from consultas;
```

```
insert into consultas(data_consulta,descricao) values('2025-8-27','teste de inserção');
```

```
select * from log_consultas;
```



Diferença entre OLD e NEW

Referências especiais para acessar dados em triggers no contexto do PlanoSaudeDB:

NEW

- Valores sendo inseridos em uma nova consulta médica
- Valores após atualização (ex: NEW.data_consulta, NEW.valor)
- Não disponível em DELETE triggers
- Usado em BEFORE/AFTER INSERT
- Usado em BEFORE/AFTER UPDATE

Exemplo: **NEW.valor** acessa o novo valor da consulta após alteração de preço

OLD

- Não disponível em INSERT triggers
- Valores antes da atualização (ex: OLD.id_medico, OLD.status)
- Valores sendo excluídos do registro de consultas
- Usado em BEFORE/AFTER UPDATE
- Usado em BEFORE/AFTER DELETE

Exemplo: **OLD.id_paciente** acessa o ID do paciente antes da exclusão do registro

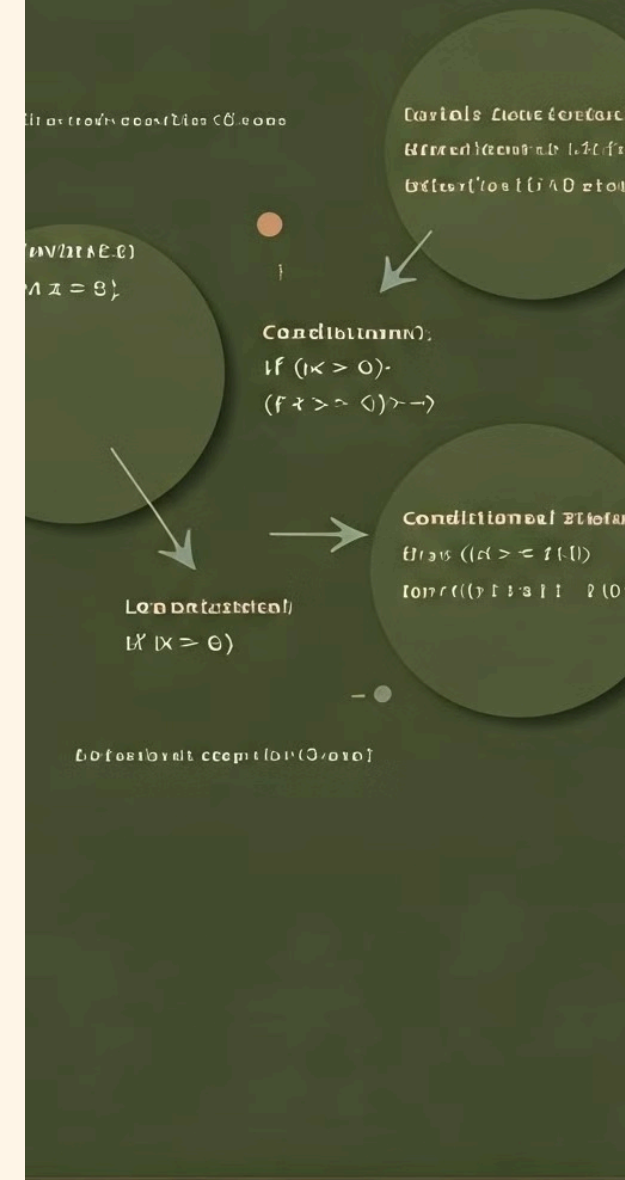
Essencial para auditoria de dados e validações no PlanoSaudeDB, permitindo comparar valores antes e depois das operações.

Explicação do Exemplo

A trigger AFTER INSERT mostrada anteriormente registra cada nova consulta na tabela de auditoria:

- O `DELIMITER` muda temporariamente o caractere que o MySQL usa para entender quando um comando termina.
- `CREATE TRIGGER trg_after_insert_consulta`: Está criando um **trigger** chamado `trg_after_insert_consulta`.
- `AFTER INSERT ON Consultas`: Esse trigger será disparado **automaticamente depois de um INSERT** na tabela `Consultas`.
- `FOR EACH ROW`: A trigger será executada **para cada linha inserida**, ou seja, se você inserir várias consultas de uma vez, o trigger roda uma vez para cada uma.
- `NEW.id_consulta`: Refere-se ao `id_consulta` da **nova linha** que foi inserida na tabela `Consultas`.
- `NOW()`: Registra a data e hora atual do sistema.
- `USER()`: Captura o nome do **usuário MySQL** que executou o comando `INSERT`.

Essa trigger garante a rastreabilidade completa de novas consultas médicas no sistema, criando um histórico de auditoria para fins de compliance e segurança.



BEFORE INSERT para Validação

OCTOBER

					1	
				7	8	
3	4	5	16	14	8	16
11	12	12	13	14	15	12
18	19	29	20	21	22	23
24	25	26	27	28	29	

```
DELIMITER //
CREATE TRIGGER trg_antes_insert_consulta
BEFORE INSERT ON Consultas
FOR EACH ROW
BEGIN
    IF NEW.data_consulta > CURDATE() + INTERVAL 1 YEAR THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Data da consulta muito distante';
    END IF;
END;
// DELIMITER ;
```

```
select * from consultas;
```

```
insert into consultas(data_consulta,descricao)
```

```
values('2028-10-25','teste de inserção');
```

Error Code: 1644. Data da consulta muito distante

```
select * from log_consultas;
```

VALIDATION FAILURE

Explicação da Validação



SIGNAL
SQLSTATE
'45000'

Lança erro personalizado com código SQL específico, interrompendo a operação de INSERT quando a data da consulta excede o limite permitido no PlanoSaudeDB



CURDATE() +
INTERVAL

Calcula a data limite permitida para agendamento (1 ano à frente), evitando reservas excessivamente antecipadas na clínica



Proteção de
Integridade

Garante que apenas datas válidas sejam inseridas na tabela Consultas, mantendo a consistência do sistema de agendamento do PlanoSaudeDB

Posso alterar uma trigger existente no MySQL?

No MySQL, **não é possível alterar diretamente uma trigger existente** com um comando tipo `ALTER TRIGGER`, como fazemos com tabelas, por exemplo.

O que dá pra fazer então?

Você tem duas opções:

1. **Excluir a trigger existente**
2. **Criar uma nova com as modificações desejadas**

Como excluir uma trigger:

```
DROP TRIGGER nome_da_trigger; Copiar
```

AFTER UPDATE: Log de atualizações

```
DELIMITER //  
CREATE TRIGGER trg_after_update_consulta  
AFTER UPDATE ON Consultas  
FOR EACH ROW  
BEGIN  
    INSERT INTO Log_Consultas(id_consulta, acao, data_log,  
usuario)  
    VALUES (NEW.id_consulta, 'UPDATE', NOW(), USER());  
END;  
// DELIMITER ;
```

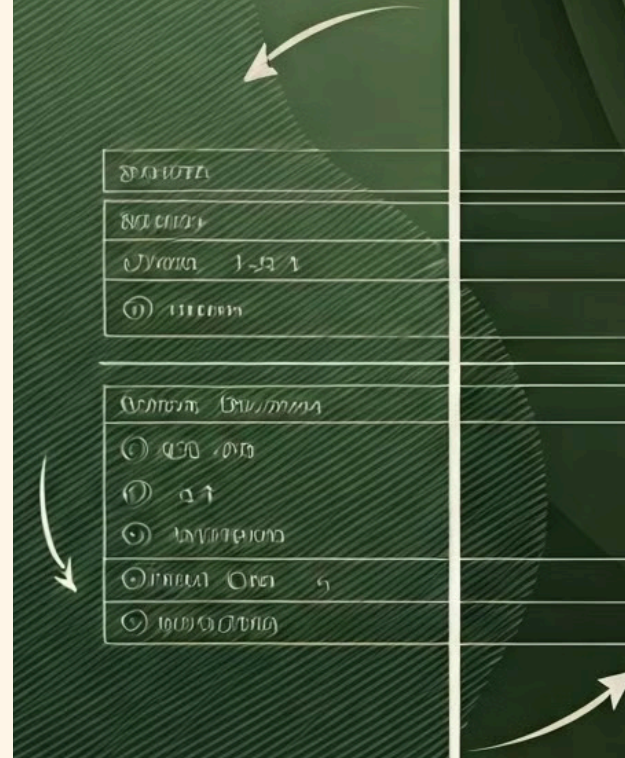
select * from consultas;

UPDATE Consultas

SET descricao = 'Descrição alterada'

WHERE id_consulta = 1;

select * from Log_Consultas;



id_consulta	acao	data_log	usuario
1	UPDATE	2023-10-10 14:10:10	root@localhost

id_consulta	descricao
1	Descrição alterada

BEFORE UPDATE para Validação



A trigger BEFORE UPDATE interrompe operações potencialmente perigosas antes que alterem o banco de dados, garantindo a integridade dos dados do PlanoSaudeDB e o cumprimento das regras de negócio da área de saúde.

Exemplo: Bloquear update se a consulta já foi autorizada

Vamos antes alterar a tabela Consultas usando ALTER TABLE:

```
ALTER TABLE Consultas  
DROP COLUMN descricao,  
ADD COLUMN status VARCHAR(20),  
ADD COLUMN autorizada BOOLEAN;
```

Trigger para bloquear a alteração

```
DELIMITER //  
CREATE TRIGGER trg_before_update_consulta  
BEFORE UPDATE ON Consultas  
FOR EACH ROW  
BEGIN  
    -- Verifica se a consulta já estava autorizada ANTES do update  
    IF OLD.autorizada = TRUE THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Consulta já autorizada pela operadora.';  
    END IF;  
END;  
// DELIMITER ;
```


Explicação

IF OLD.autorizada = TRUE THEN

- OLD representa os **valores antigos** da linha (antes da atualização).
- Aqui, ele verifica se o campo `autorizada` da consulta já estava `TRUE` (ou seja, já foi autorizada pela operadora).

SIGNAL SQLSTATE '45000'

- Este comando gera **um erro personalizado** no MySQL.
- '45000' é um código SQLSTATE genérico para **erros definidos pelo usuário**.

SET MESSAGE_TEXT = '...'

- Define a mensagem que será exibida quando o erro for gerado.
- Neste caso: **"Consulta já autorizada pela operadora."**

Testando a trigger before update

Insira um registro:

```
INSERT INTO
Consultas (id_consulta, data_consulta, status, autorizada)
VALUES (1, '2025-04-25', 'Agendada', FALSE);
```

Agora autorize:

```
UPDATE Consultas
SET autorizada = TRUE
WHERE id_consulta = 2;
```

Depois tente alterar algo **depois de autorizado**:

```
UPDATE Consultas
SET status = 'Concluída'
WHERE id_consulta = 2;
```

Error Code: 1644. Consulta já autorizada pela operadora.

```
UPDATE Consultas

SET autorizada = FALSE

WHERE id_consulta = 2;
```

Error Code: 1644. Consulta já autorizada pela operadora.

Triggers e DELETE

```
delimiter //  
CREATE TRIGGER trg_after_delete_consulta  
AFTER DELETE ON Consultas  
FOR EACH ROW  
BEGIN  
    INSERT INTO Log_Consultas(id_consulta, acao, data_log, usuario)  
    VALUES (OLD.id_consulta, 'DELETE', NOW(), USER());  
END;  
// delimiter ;
```

INSERT INTO

Consultas (id_consulta, data_consulta, status, autorizada)

VALUES (1, '2025-04-25', 'Agendada', FALSE);

delete from consultas where id_consulta = 1;

select * from Log_Consultas;



Vantagens



Automatização

Regras de negócio aplicadas automaticamente no momento da operação DML, como validação de valores mínimos para consultas no PlanoSaudeDB ou verificação de disponibilidade de médicos



Segurança

Maior consistência dos dados através de validações antes da inserção/atualização, garantindo que registros de exames e consultas sempre tenham valores dentro dos parâmetros permitidos pelo plano de saúde



Menos redundância

Reduz código na aplicação cliente ao centralizar regras no banco de dados, evitando que cada sistema que acesse o PlanoSaudeDB precise reimplementar validações de datas, valores ou relacionamentos entre pacientes e médicos



Desvantagens



Complexidade

Maior dificuldade na manutenção do schema do PlanoSaudeDB, especialmente quando múltiplas triggers interagem entre tabelas relacionadas como Consultas, Exames e Pacientes.

Depuração

Erros em triggers são complexos de rastrear porque ocorrem em segundo plano durante operações DML no PlanoSaudeDB, sem mensagens claras para o usuário final.

Desempenho

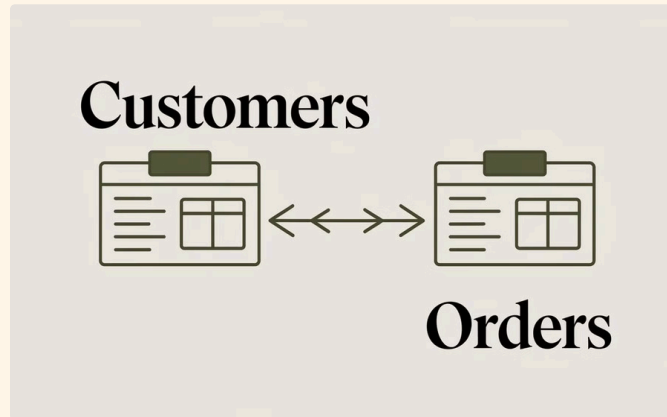
Triggers como `trg_after_insert_exame` podem impactar negativamente a performance de inserções em massa ou atualizações frequentes nas tabelas de Exames e Consultas do PlanoSaudeDB.

Evitando armadilhas



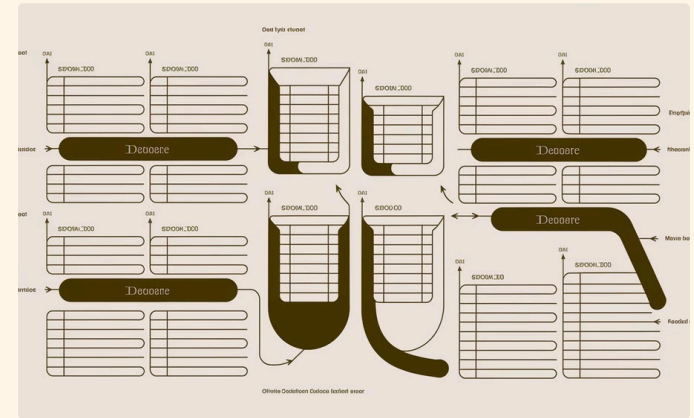
Lógica simples

Mantenha triggers com menos de 20 linhas para facilitar manutenção. Triggers complexas como validação de coberturas no PlanoSaudeDB podem gerar erros difíceis de rastrear.



Independência

Evite triggers que chamam triggers em cascata entre tabelas Consultas e Exames. Ciclos de dependência podem causar deadlocks e comprometer a integridade do PlanoSaudeDB.



Organização

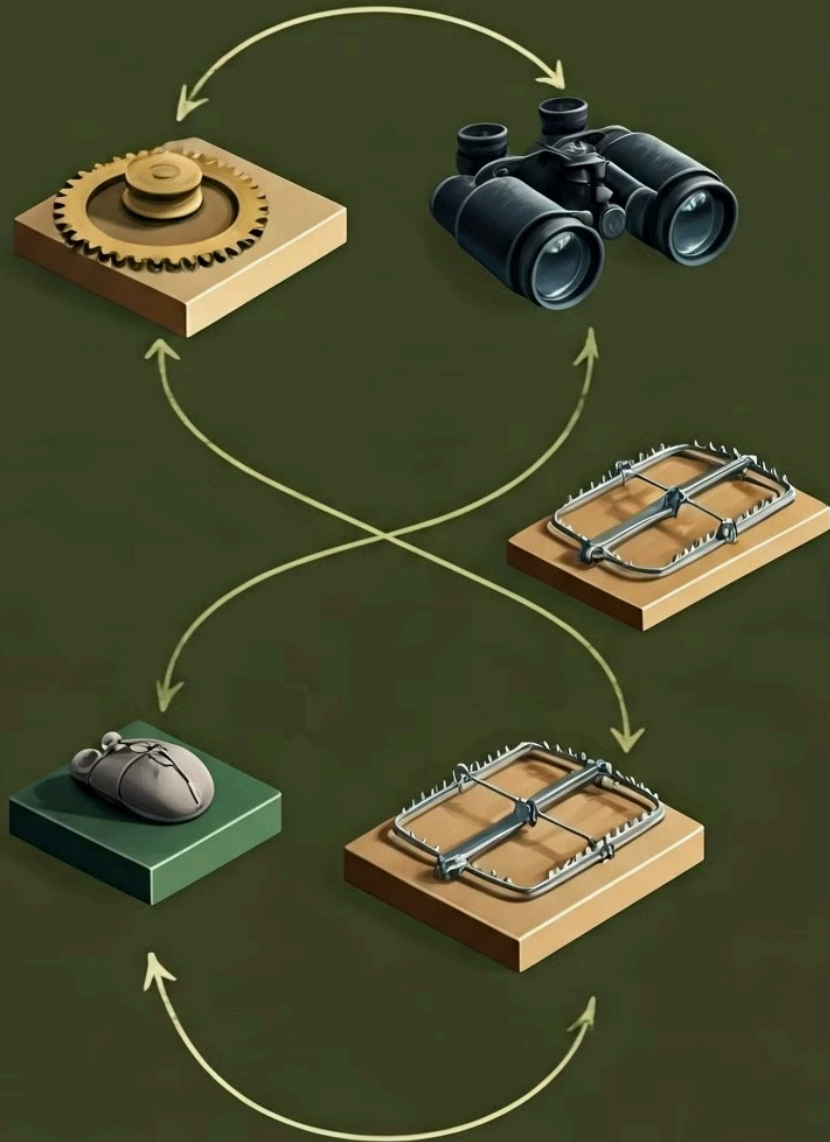
Documente cada trigger com propósito, tabelas afetadas e possíveis efeitos colaterais. Use prefixos padronizados como `trg_before_insert_consulta` para fácil identificação no PlanoSaudeDB.



Verificando Triggers Existentes

```
SHOW TRIGGERS FROM PlanoSaudeDB;
```


Comparando com Procedures e Views



Recurso	Acionamento	Finalidade
Procedures	Chamada explícita pela aplicação do PlanoSaudeDB	Processamento em lote de faturamento e relatórios médicos
Views	Consulta direta pelos operadores do sistema	Relatórios pré-formatados de pacientes e cobertura de planos
Triggers	Eventos DML nas tabelas Consultas, Exames e Pacientes	Validação automática de cobertura e auditoria de alterações

Quando usar e quando evitar

Use para:

- Validações básicas de datas de consultas e valores
- Auditoria de alterações em registros de pacientes
- Verificações de cobertura do plano em exames
- Atualizações automáticas de valores com desconto
- Controle de integridade entre tabelas do PlanoSaudeDB

Evite para:

- Cálculos complexos de faturamento mensal
- Processos que dependem de múltiplas tabelas
- Lógica de negócio que muda frequentemente
- Operações com acesso a sistemas externos
- Rotinas de envio de notificações a pacientes